

| **ml.a8**

| **workout**

Images (Raw) $\rightarrow$ Grayscale + Resize

$\downarrow$
Normalization

$\downarrow$
Convolution - layers

$\downarrow$
Max pooling (dim reduce)

$\downarrow$
Flattening

$\downarrow$
Input to FCN

$\downarrow$
sigmoid out (binary)

$\downarrow$
Loss fn (BCE)

$\downarrow$
Backprop (update wts)

Network Architecture

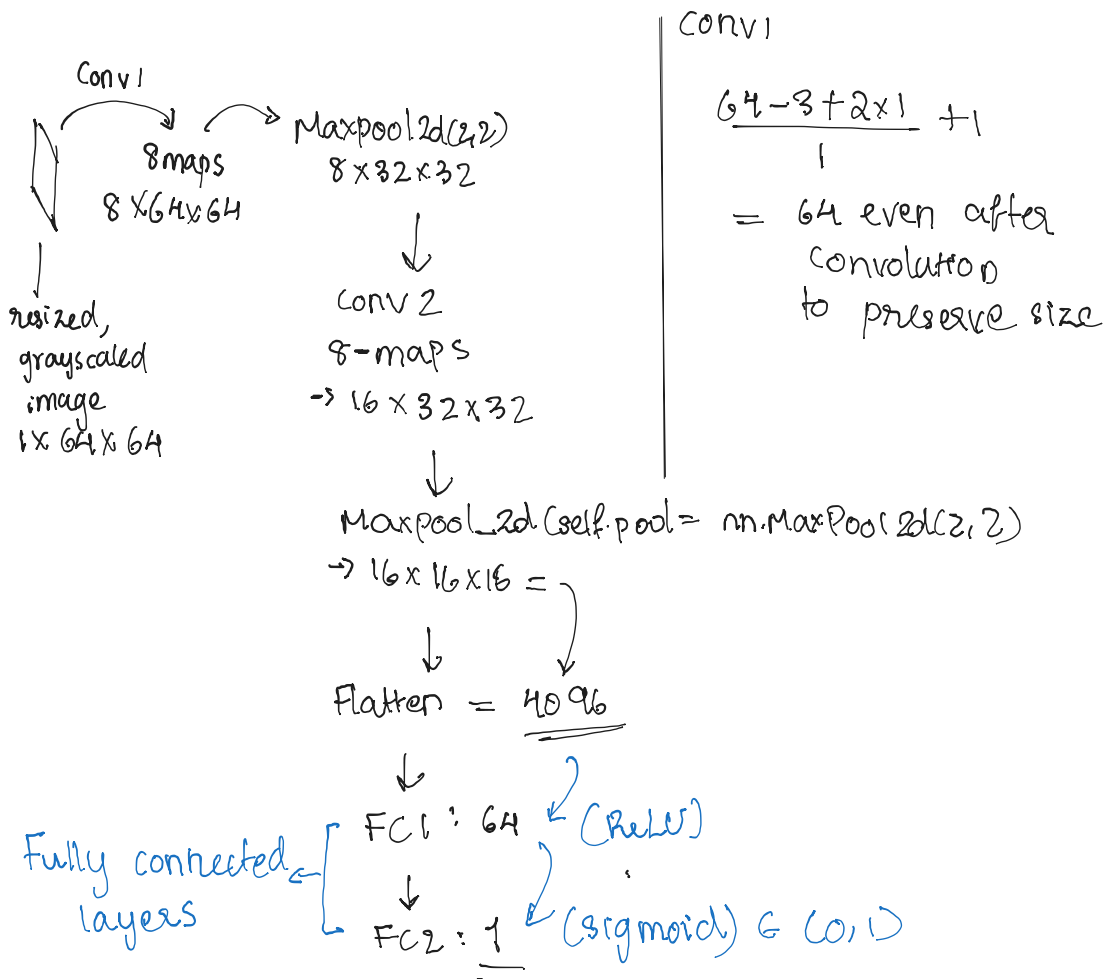


Image transformation part:

$n = \text{no. of images}$

- 1) $(H, W, 1) \rightarrow (1, H, W)$
 $\sim \text{conv2d}(1, 8, 8)$
 - 2) $255 \times 255 \rightarrow 64 \times 64$ (Resize)
 - 3) $\text{toTensor}()$
 $[0, 255] \rightarrow [0, 1]$
 - 4) $\text{transform.Normalize}(\mu=0.5, \sigma=0.5)$
 $x' = \frac{x - 0.5}{0.5} \quad [0, 1] \rightarrow [-1, 1]$
 - 5) $\text{Batch-size} = \text{all} (140)$
 $\hat{u} (140, 1, 64, 64)$
-

- Padding of 1 with filter size 3×3 will output same size

Previous FCN model:

Input: 784 neurons (28×28) flattened

H1 $\rightarrow$ 128 neurons + ReLU

H2 $\rightarrow$ 64 neurons + ReLU

output $\rightarrow$ 10 neurons + softmax

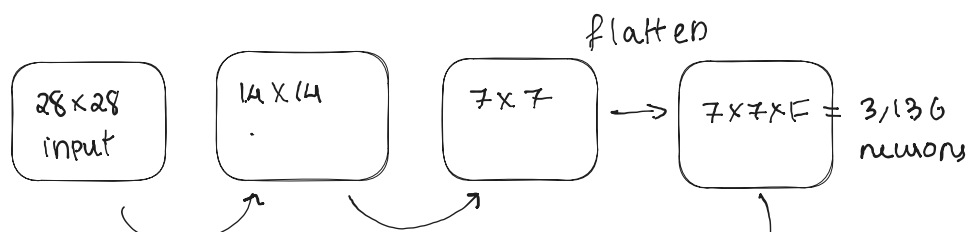
Backpropagation

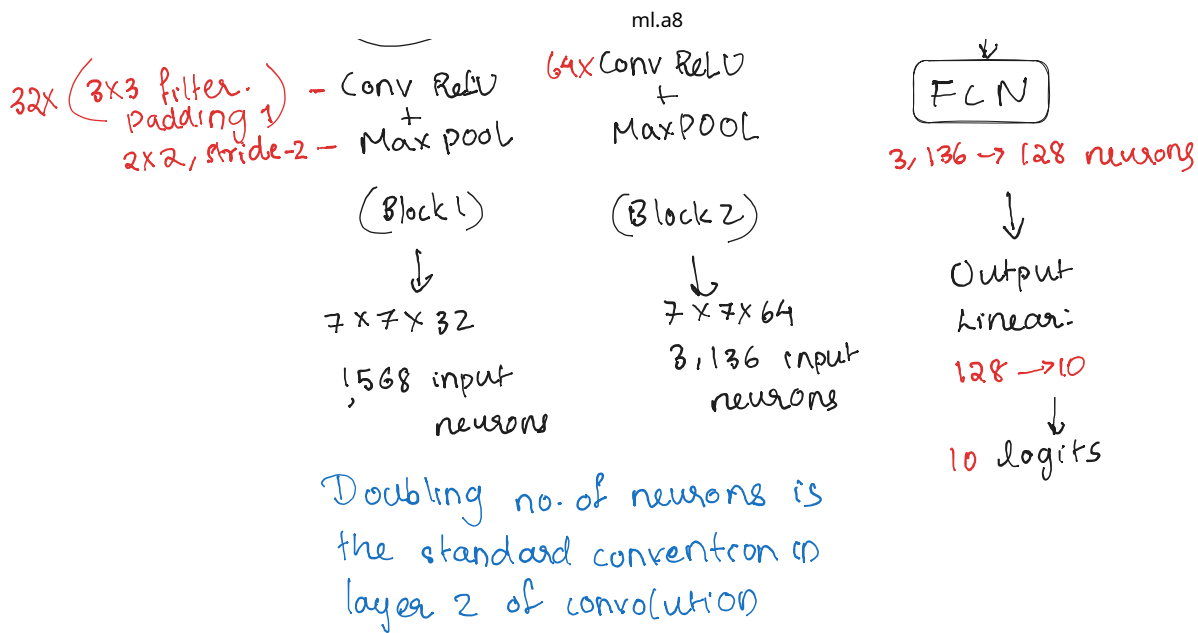
$\sim \text{acc} = 78.47$.

- 1) $\text{toTensor}()$: converts 0-255 into 0.0 $\rightarrow$ 1.0
 $\downarrow$
 $\text{Normalize}(\mu, \sigma)$ for MNIST $\mu = 0.1307$
 $\sigma = 0.3081$

Dset = (60k - Training + 10k test) images

batch-size = 64 $\rightarrow 2^6$





- Gradients are stored in the model's params & optimizers references to those parameters

optimizer.zero_grad() - clears old grad

↓

forward pass

↓

compute loss

↓

compute gradients

↓

update wts.

Compute behavior over epochs

model.eval() → changes model behavior

torch.no_grad() → changes memory / compute behavior